Experiment 7

Student Name: Diwansh
UID: 24BAI70294
Branch: CSE - AIML
Section/Group: 24AIT_KRG G1
Semester: 4
Date of Performance: 13/3/26
Subject Name: DBMS
Subject Code: 24CSH-298

Aim

To design and implement simple views, complex views, and materialized views, and to compare and analyse execution time and performance differences between them using query analysis tools.

Software Requirements

Database Management System:

• Oracle Database Express Edition (Oracle XE)
• PostgreSQL Database

Database Administration Tool / Client Tool:

• Oracle SQL Developer (for Oracle XE)
• pgAdmin (for PostgreSQL)

Objectives

• To create simple views, complex views, and materialized views.
• To perform JOIN and aggregation operations on relational tables.
• To compare query performance using EXPLAIN ANALYZE.
• To understand the advantages of materialized views in improving query performance.

Practical/Experiment Steps

• **Relational Schema Construction:** Created the depts and emps tables with a primary-foreign key relationship to represent department-employee structure.

• **Simple View Implementation:** Created a simple view (V_SIMPLE) to display employee names and salaries where salary is greater than a specific threshold.

• **Complex View Implementation:** Created a complex view (V_COMPLEX) using JOIN, GROUP BY, and aggregate functions such as SUM() and AVG() to calculate departmental salary statistics.

• **Materialized View Creation:** Created a materialized view (V_MATERIALIZED) using the same logic as the complex view to store computed results physically.

• **Performance Analysis:** Used EXPLAIN ANALYZE to evaluate execution plans and compare performance of simple view, complex view, and materialized view.

• **Data Synchronization:** Used REFRESH MATERIALIZED VIEW to update the materialized view with latest data from base tables.

Procedure

• Opened the PostgreSQL environment using pgAdmin and connected to the database.

• Created the depts and emps tables and inserted sample data.

• Created a simple view to filter employees based on salary.

• Created a complex view using JOIN and GROUP BY to calculate department-wise salary totals and averages.
• Created a materialized view to store the same aggregated data physically.
• Executed SELECT queries on all views to verify results.
• Ran EXPLAIN ANALYZE on each view to compare execution time and query plans.
• Refreshed the materialized view using REFRESH command.
• Analysed the performance difference between virtual views and materialized views.

Input/Output Analysis

SQL Input Queries

```
CREATE TABLE depts(
dept_id INT PRIMARY KEY,
dept_name VARCHAR(20)
);

CREATE TABLE emps(
emp_id INT PRIMARY KEY,
name VARCHAR(20),
dept_id INT REFERENCES depts(dept_id),
salary NUMERIC
);
```

Output

```
SQL> CREATE TABLE emps(
     emp_id INT PRIMARY KEY,
     name VARCHAR(20),
     dept_id INT REFERENCES depts(dept_id),...
Show more...



Table EMPS created.

Elapsed: 00:00:00.011
```

SQL Input Queries

INSERT INTO depts VALUES(1, 'IT'), (2, 'HR'), (3, 'SALES');

INSERT INTO emps VALUES(101, 'Mary', 1, 95000);
INSERT INTO emps VALUES(102, 'Amit', 1, 85000);
INSERT INTO emps VALUES(103, 'Sarah', 2, 70000);
INSERT INTO emps VALUES(104, 'John', 2, 65000);
INSERT INTO emps VALUES(105, 'Jack', 3, 55000);
INSERT INTO emps VALUES(106, 'Rohan', 1, 88000);

Output

```
SQL> INSERT INTO emps VALUES(101, 'Mary', 1, 95000)



1 row inserted.

Elapsed: 00:00:00.013
```

```
SQL> INSERT INTO emps VALUES(102, 'Amit', 1, 85000)



1 row inserted.

Elapsed: 00:00:00.001
```

```
SQL> INSERT INTO emps VALUES(103, 'Sarah', 2, 70000)



1 row inserted.

Elapsed: 00:00:00.002
```

SQL Input Queries

CREATE VIEW V_SIMPLE AS
SELECT name, salary FROM emps WHERE salary > 75000;

SELECT * FROM V_SIMPLE;

Output

	NAME	SALARY
1	Mary	95000
2	Amit	85000
3	Rohan	88000

SQL Input Queries

```
CREATE VIEW V_COMPLEX AS
SELECT d.dept_name, SUM(e.salary) AS total_budget, AVG(e.salary) AS avg_sal
FROM emps e JOIN depts d
ON e.dept_id = d.dept_id
GROUP BY d.dept_name;

SELECT * FROM V_COMPLEX;

SELECT * FROM V_COMPLEX;
```

Output

	DEPT_NAME	TOTAL_BUDGET	AVG_SAL
1	IT	268000	89333.33333333333
2	HR	135000	67500
3	SALES	55000	55000

SQL Input Queries

```
CREATE MATERIALIZED VIEW V_MATERIALIZED AS
SELECT d.dept_name, SUM(e.salary) AS total_budget, AVG(e.salary) AS avg_sal
FROM emps e JOIN depts d
ON e.dept_id = d.dept_id
GROUP BY d.dept_name;

SELECT * FROM V_MATERIALIZED;
```

Output

	DEPT_NAME	TOTAL_BUDGET	AVG_SAL
1	IT	268000	89333.33333333333
2	HR	135000	67500
3	SALES	55000	55000

SQL Input Queries

```
REFRESH MATERIALIZED VIEW V_MATERIALIZED;
```

Output

Data Output Messages Notifications

```
REFRESH MATERIALIZED VIEW

Query returned successfully in 175 msec.
```

SQL Input Queries

EXPLAIN ANALYZE SELECT * FROM V_SIMPLE;

Output

	QUERY PLAN text
1	Seq Scan on emps (cost=0.00..18.00 rows=213 width=90) (actual time=0.018..0.021 rows=3.00 loop...
2	Filter: (salary > '75000'::numeric)
3	Rows Removed by Filter: 3
4	Buffers: shared hit=1
5	Planning Time: 0.101 ms
6	Execution Time: 0.033 ms

SQL Input Queries

EXPLAIN ANALYZE SELECT * FROM V_COMPLEX;

Output

	QUERY PLAN text
1	HashAggregate (cost=51.54..54.54 rows=200 width=122) (actual time=0.110..0.117 rows=3.00 loops=1)
2	Group Key: d.dept_name
3	Batches: 1 Memory Usage: 32kB
4	Buffers: shared hit=2
5	-> Hash Join (cost=30.25..48.34 rows=640 width=90) (actual time=0.081..0.089 rows=6.00 loops=1)
6	Hash Cond: (e.dept_id = d.dept_id)
7	Buffers: shared hit=2
8	-> Seq Scan on emps e (cost=0.00..16.40 rows=640 width=36) (actual time=0.036..0.037 rows=6.00 loops=1)
9	Buffers: shared hit=1
10	-> Hash (cost=19.00..19.00 rows=900 width=62) (actual time=0.036..0.036 rows=3.00 loops=1)
11	Buckets: 1024 Batches: 1 Memory Usage: 9kB
12	Buffers: shared hit=1
13	-> Seq Scan on depts d (cost=0.00..19.00 rows=900 width=62) (actual time=0.025..0.027 rows=3.00 loo...
14	Buffers: shared hit=1
15	Planning Time: 0.558 ms
16	Execution Time: 0.184 ms

SQL Input Queries

EXPLAIN ANALYZE SELECT * FROM V_MATERIALIZED;

Output

	QUERY PLAN text
1	Seq Scan on v_materialized (cost=0.00..15.40 rows=540 width=122) (actual time=0.026..0.027 rows=3.00 loo...
2	Buffers: shared hit=1
3	Planning Time: 0.090 ms
4	Execution Time: 0.041 ms

Learning Outcomes

• Gained understanding of simple views, complex views, and materialized views.

• Learned how JOIN and aggregate functions work in real-world scenarios.

• Understood how EXPLAIN ANALYZE helps in evaluating query performance.

• Learned the importance of materialized views in improving query execution speed by storing precomputed results.